Free Form Testing Agent

Document: 06_recovery_system.md

Version: 1.0
Status: Implemented Architecture
Purpose: Explain the Interrupted Session Recovery System
Last Updated: July 2026

---

1. Introduction

The Recovery System is one of the most important subsystems in the Free Form Testing Agent.

Its purpose is very simple:

> **If the testing process stops unexpectedly, the framework should continue exactly where it left off instead of starting from the beginning.**

Without a recovery system, a long-running exploration could lose hours of work because of:

- Power failure
- PC restart
- Process crash
- Manual termination
- IDE crash
- Operating system update

The Recovery System ensures that the framework **remembers everything it has already discovered**.

---

2. The Problem It Solves

Imagine the explorer has already discovered:

```
States:
150

Transitions:
280
```

```
Explored Actions:
920
```

After 4 hours of exploration, the computer suddenly shuts down.

Without recovery:

```
Restart Framework

↓

Everything starts from State S0

↓

All 4 hours of work are lost
```

With recovery:

```
Restart Framework

↓

Load Previous Session

↓

Restore Graph

↓

Restore Memory

↓

Continue from Action 921
```

Nothing important is lost.

---

3. What Recovery Actually Means

Many people confuse **Recovery** with **Replay**.

They are completely different.

Recovery

Recovery rebuilds the **framework's knowledge**.

```
Disk
   ↓
Runtime Objects
```

It restores:

- Graph
- Memory
- Session
- Statistics
- Pending targets

Replay

Replay restores the **application**.

```
Graph
   ↓
Application State
```

It physically drives the application back to a required state.

Easy Way to Remember

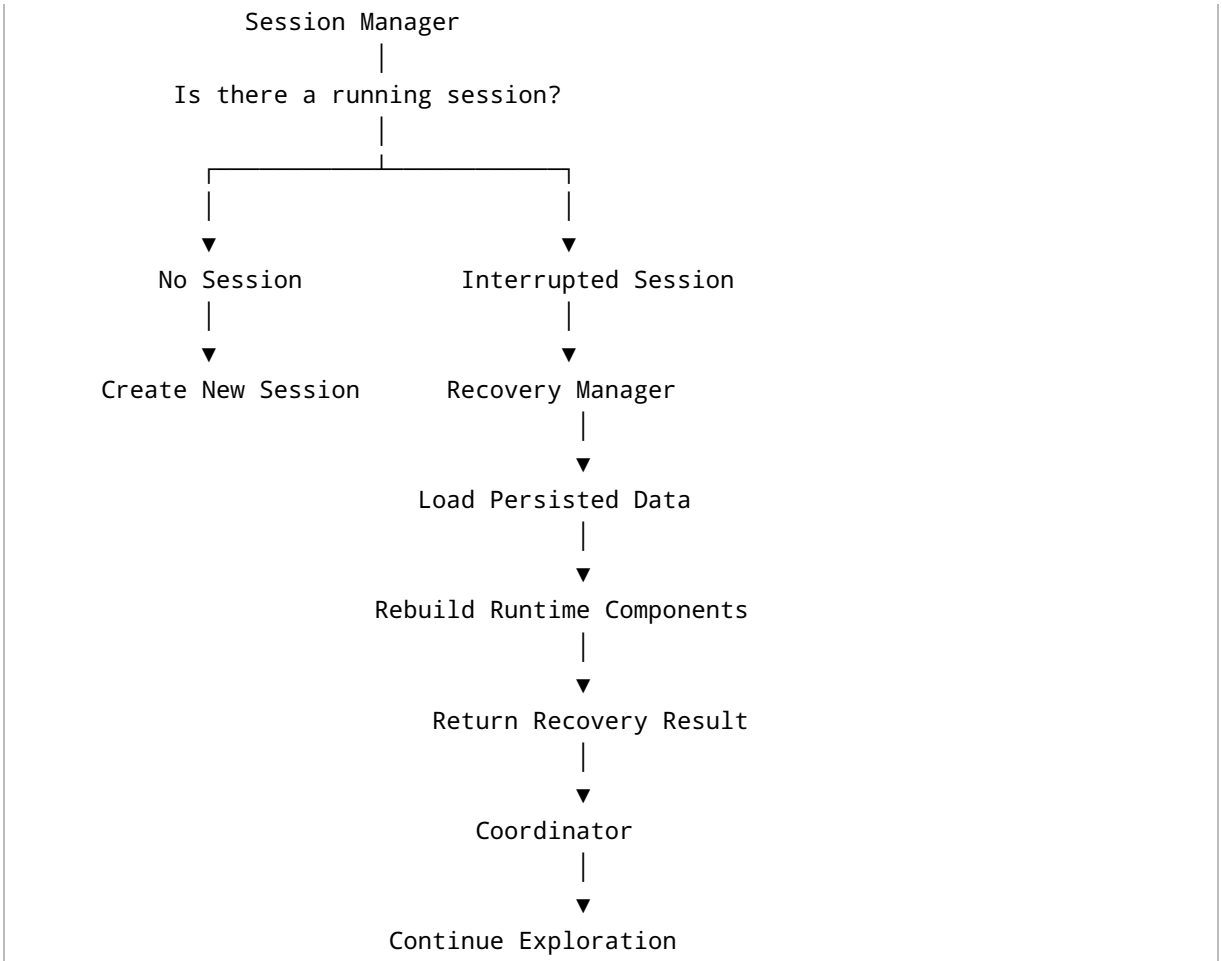
Recovery restores **the brain**.

Replay restores **the body**.

---

4. High-Level Architecture

```
                Framework Starts
                       │
                       ▼
```



---

5. Responsibilities of the Recovery System

The Recovery System has only one responsibility:

> **Reconstruct the runtime exactly as it existed before interruption.**

It does **not**:

- Replay UI actions
- Execute application actions
- Explore new states
- Detect bugs

Those responsibilities belong to other components.

---

6. Recovery Workflow

The complete recovery flow looks like this:

```
Framework Starts

↓

Load Session

↓

Find Interrupted Session

↓

Load State Graph

↓

Load Exploration Memory

↓

Load Statistics

↓

Load Pending Targets

↓

Rebuild Runtime Objects

↓

Return Recovery Result

↓

Coordinator Continues
```

Notice that the application has not yet been restored.

Only the framework has.

7. Recovery Components

The recovery subsystem consists of several small components.

```
Recovery Manager
        │
        ├── Session Loader
        ├── Graph Loader
        ├── Memory Loader
        ├── Statistics Loader
        ├── Validation
        └── Recovery Result
```

Keeping these responsibilities separate makes the system easier to test.

---

8. Main Recovery Objects

8.1 Exploration Session

Represents one complete testing run.

Example:

```
ExplorationSession(
    id="session-001",
    status="RUNNING",
    started_at="10:15",
    explored_states=124
)
```

Purpose:

Tracks the lifetime of exploration.

8.2 State Graph

Represents discovered application behavior.

Example:

```
S0 --7--> S1

S1 --+--> S2

S2 --3--> S3
```

Purpose:

Stores application knowledge.

8.3 Exploration Memory

Stores exploration progress.

Example:

```
ExplorationMemory(
    visited_states={S0,S1,S2},
    explored_actions={
        (S0,7),
        (S1,+)
    }
)
```

Purpose:

Remember what work has already been completed.

8.4 Exploration Target

Represents unfinished work.

Example:

```
State S5

Remaining Actions:

=
```

```
*

/
```

Purpose:

Allows exploration to continue from unfinished areas.

8.5 Recovery Result

The Recovery Manager returns one object.

Example:

```
RecoveryResult(
    session=...,
    graph=...,
    memory=...,
    statistics=...
)
```

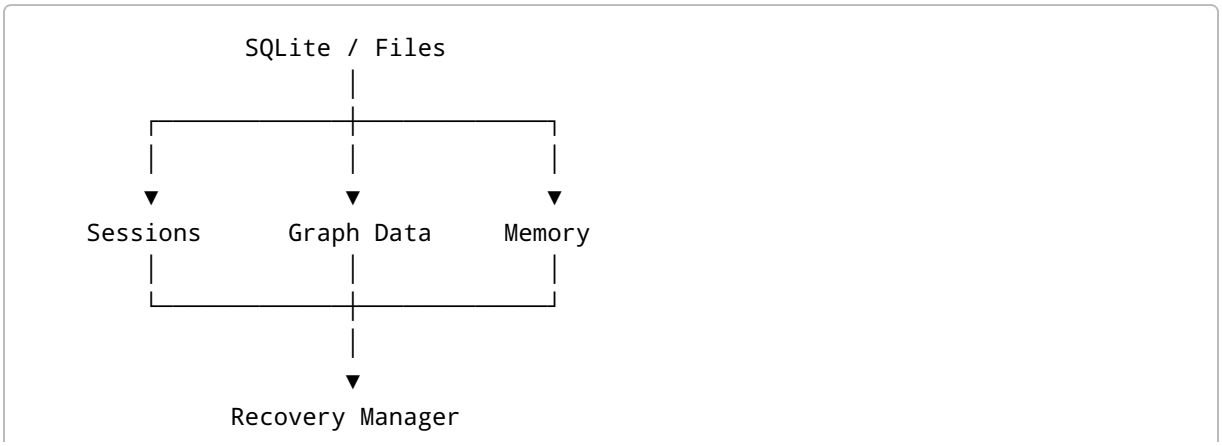
Purpose:

Provide everything needed to restart exploration.

---

9. Recovery Data Flow

This is the most important diagram.



```
                  |
          Validate Loaded Data
                  |
                  ▼
             Recovery Result
                  |
                  ▼
         Exploration Coordinator
                  |
                  ▼
        Continue Exploration
```

Notice:

No application interaction happens here.

Only framework reconstruction.

---

10. Runtime Data Before Recovery

Imagine the explorer was running.

Memory:

```
Visited States:

S0

S1

S2
```

Graph:

```
S0 → S1

S1 → S2
```

Session:

```
RUNNING
```

Everything is periodically persisted.

---

11. Crash Happens

Suppose the process dies.

```
Runtime Objects

❌ Lost
```

But disk still contains:

```
Session

Graph

Memory

Screenshots

Statistics
```

This is why persistence exists.

---

12. Startup After Crash

When the framework starts again:

```
Load Latest Session

↓

Status == RUNNING ?

↓
```

```
YES
```

A RUNNING session means:

```
Previous execution never completed.
```

Therefore:

Recovery begins.

---

13. Recovery Sequence

The Recovery Manager performs these steps.

Step 1

Load session.

↓

Step 2

Load graph.

↓

Step 3

Load memory.

↓

Step 4

Load statistics.

↓

Step 5

Validate.

↓

Step 6

Build runtime.

↓

Step 7

Return RecoveryResult.

---

14. Data Transformation

Before persistence:

```
Runtime Objects
```

During persistence:

```
Serialize

↓

SQLite

↓

Files
```

During recovery:

```
SQLite

↓

Deserialize

↓
```

```
Runtime Objects
```

Recovery is essentially the reverse operation of persistence.

---

15. Validation During Recovery

Recovery should never blindly trust disk.

Example checks:

```
Does Session Exist?

Does Graph Exist?

Does Memory Exist?

Are IDs Valid?

Are References Valid?
```

If something is corrupted:

Recovery should fail safely.

---

16. Recovery States

Recovery itself has states.

```
STARTING

↓

LOADING

↓

VALIDATING

↓
```

```
REBUILDING

↓

READY

↓

FAILED
```

This makes debugging easier.

---

17. Recovery Manager Interface

Conceptually:

```
class RecoveryManager:

    def recover(self):

        ...

        return RecoveryResult
```

The coordinator does not care how recovery works internally.

It only needs the result.

---

18. Recovery Result

The Recovery Result contains everything needed to continue.

Example:

```
RecoveryResult(

    session=...,

    graph=...,
```

```
    memory=...,

    statistics=...,

    recovered=True
)
```

The coordinator receives one object.

No additional loading is required.

---

19. Coordinator Integration

Without recovery:

```
Create Graph

Create Memory

Create Session

Start Exploration
```

With recovery:

```
Recover Runtime

↓

Coordinator Receives Objects

↓

Continue Exploration
```

The coordinator doesn't need special logic.

It simply receives initialized runtime objects.

---

20. Relationship with Replay

Recovery finishes here:

```
Recovered Runtime
```

Replay starts afterwards.

```
Recovered Runtime

↓

Target Selector

↓

Replay

↓

Application Restored

↓

Continue Exploration
```

Recovery and Replay never overlap.

---

21. Relationship with Process Health Check

After recovery:

```
Runtime Rebuilt
```

Before replay:

```
Health Check

↓
```

```
Application Alive?

↓

YES

Continue

↓

NO

Restart Application
```

Recovery rebuilds knowledge.

Health Check verifies the environment.

---

22. Relationship with Persistence

Persistence writes.

Recovery reads.

```
Persistence

Runtime

↓

Disk



Recovery

Disk

↓

Runtime
```

They are opposite operations.

23. Common Misunderstandings

Misunderstanding 1

Recovery restores the application.

❌ False

Replay restores the application.

Misunderstanding 2

Recovery executes UI actions.

❌ False

Executor performs actions.

Misunderstanding 3

Recovery discovers states.

❌ False

Explorer discovers states.

Misunderstanding 4

Recovery decides where to continue.

❌ False

Target Selector decides that.

---

24. Advantages

The recovery system provides:

- No loss of exploration progress

- Long-running exploration support
- Crash resilience
- Automatic continuation
- Better debugging
- Reproducible sessions
- Stable experimentation

---

25. Example

Suppose exploration stopped here:

```
Graph

S0

↓

S1

↓

S2

↓

S3
```

Pending target:

```
S2

Remaining Action:

*
```

Persistence saved:

- Session
- Graph
- Memory

Later:

```
Framework Starts

↓

Recovery

↓

Coordinator

↓

Replay to S2

↓

Execute *

↓

Continue
```

The explorer never starts from S0 again.

---

26. Complete Recovery Timeline

```
Exploration Running
        │
        ▼
Persistence Checkpoint
        │
        ▼
Process Crash
        │
        ▼
Runtime Lost
        │
        ▼
Framework Restart
        │
        ▼
Recovery Manager
        │
```

```
        ▼
Load Session
        │
        ▼
Load Graph
        │
        ▼
Load Memory
        │
        ▼
Validate Data
        │
        ▼
Build Runtime Objects
        │
        ▼
Return Recovery Result
        │
        ▼
Coordinator
        │
        ▼
Health Check
        │
        ▼
Replay
        │
        ▼
Continue Exploration
```

---

27. Recovery vs Replay vs Health Check

Component	Restores	Uses
Recovery	Framework runtime	Persisted data
Replay	Application state	State graph
Health Check	Application readiness	Running process

Think of it this way:

```
Recovery
↓
```

```
Restore the explorer's memory.



Replay
↓
Restore the application's position.



Health Check
↓
Make sure the application is healthy enough to continue.
```

---

28. Mental Model

The Recovery System is the framework's ability to **remember**.

Imagine a human explorer mapping a cave.

Without recovery:

```
Explorer faints.

↓

Wakes up.

↓

Throws away the map.

↓

Starts again.
```

With recovery:

```
Explorer faints.

↓

Wakes up.

↓

Picks up the existing map.

↓

Continues walking.
```

That is exactly what the Recovery System does.

It restores **knowledge**, not **location**.

The location is restored later by the Replay System.

---

29. One Sentence Summary

```
Persistence remembers.

Recovery rebuilds.

Health Check verifies.

Replay restores.

Coordinator continues.
```